



SAP Application Development (S/4 HANA)

SAPUI5

Motivation

This chapter will show you how to develop SAPUI5 applications and demonstrate how the Model-View-Controller (MVC) concept works. Therefore, you will learn how to implement SAPUI5 applications using XML and JavaScript, and how they can be deployed to the SAP backend. Furthermore, you will work with different view elements.

Prerequisites

Before starting this exercise, you should be familiar with the Eclipse IDE. Furthermore, it is recommended that you have completed the *HTML5, CSS & JavaScript* chapter.

Content

This chapter explains the development process of an SAPUI5 application. First, it will be explained how you create an SAPUI5 project. Second, an initial view and its logic will be implemented. Afterwards, a second view is created, which is linked to the first view and shows data values which were passed from the initial view. Then, the application will be deployed to the SAP backend. Finally, it will be explained how you use a debugger to retrace the program code of SAPUI5 applications.

Lecture notes

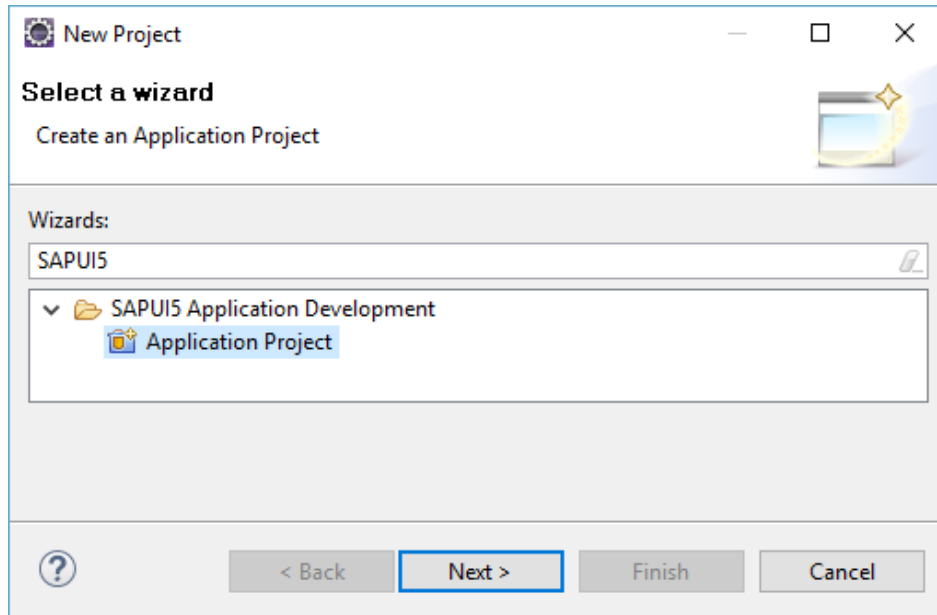
If you skipped the previous chapters on ABAP programming, your students may need to enter a developer key when they deploy their applications to the SAP backend. Please consult the lecture notes for further assistance on this topic. Furthermore, students should use their individual three-digit number when they deploy their applications to the SAP backend. In this exercise, these are displayed as ### and have to be replaced by the students' individual numbers. In addition please replace \$\$ with your two-digit institution id.

- **Product:** All
- **Level:** Advanced
- **Focus:** Programming
- **Version:** 2.0
- **Author:** UCC Technical University of Munich

Task 1: Create an SAPUI5 application project

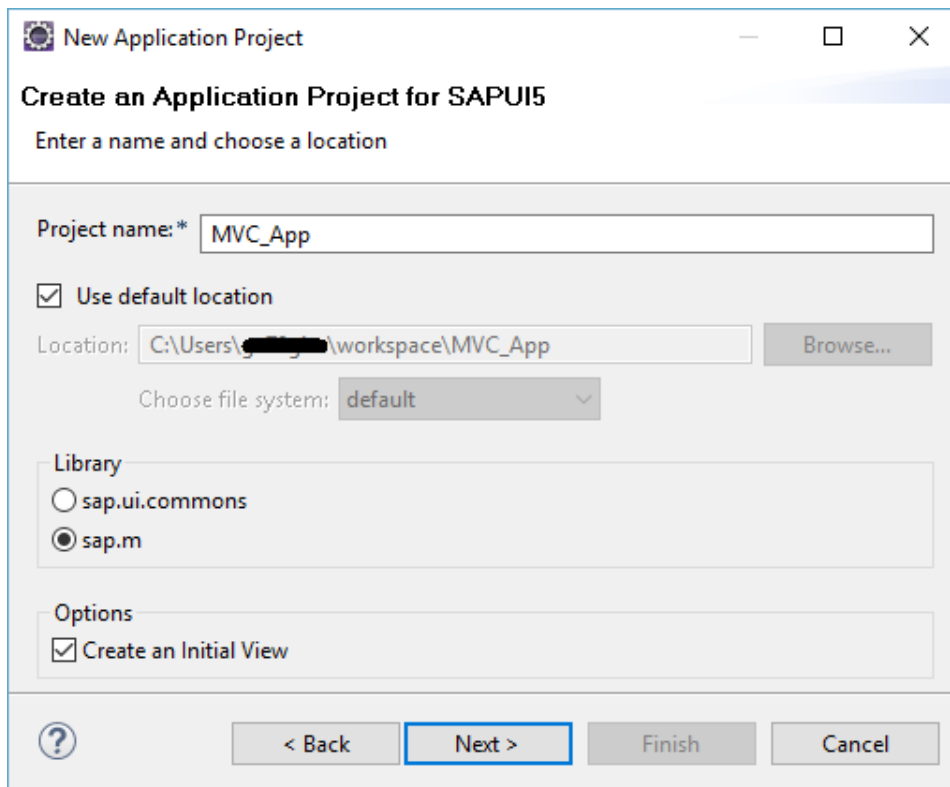
Short description: Creating an SAPUI5 application project with an initial view and controller

Start Eclipse. In the context menu of the Project Explorer, choose **New->Other->SAPUI5 Application Development->Application Project**. Click **Next**.



Create SAPUI5 application

Give your project the name **MVC_App** (no need to add your three-digit number at this point). Choose **sap.m** as library, check **Create an Initial View**, and click **Next**.



On the next page, you have to define the properties for your initial view. Enter the name **main**, and select to develop the view using **XML**. Click on **Finish** to create your project.

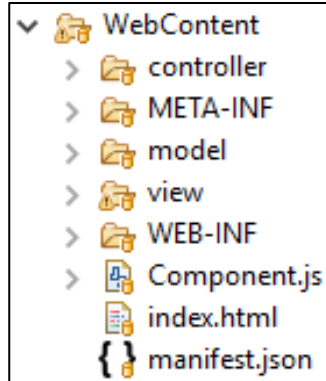
If you are asked if you want to change your perspective, click **Yes** (the recommended perspective is JAVA EE).

Task 2: Modularize your application

Short description: Make your application fit for reuse in other applications and the Fiori launchpad.

Modularizing SAPUI5-Projects enables us to reuse certain parts of an application or work on them independently. For this purpose, the concept of components comes in handy.

To turn your application into a component, recreate the folder structure on the right: inside of the WebContent folder create the folders **model**, **view** and **controller**, move your **main.view.xml** and **main.controller.js** into the corresponding folder, and delete the folder **mvc_app** (rename **Main** to **main** if necessary).



Create a file **Component.js**. We will store the information to start our application in it. Also create the file **manifest.json**. Eclipse will highlight the file as erroneous – ignore that for now. In this file we will store metadata and routing information.

```
jQuery.sap.declare("mvc_app_000.Component");
sap.ui.define([
    "sap/ui/core/UIComponent"
], function (UIComponent) {
    "use strict";

    return UIComponent.extend("mvc_app_000.Component", {
        metadata: {
            manifest: "json"
        },
        init: function () {
            UIComponent.prototype.init.apply(this, arguments);
        }
    });
});
```

Write the code on the left into the **Component.js** file. The **sap.ui.define**-statement enables us to implement the asynchronous module definition. In asynchronous module definition, files are only loaded when they are needed and not all at once, improving the overall performance. The first parameter in the statement is an array, containing all the dependencies required. The second parameter is a function, returning the respective resource, that has been defined. We extend the **UIComponent**-Class, passing the name of our module (namespace + **.Component**). In this case we choose the namespace **mvc_app** which is further specified in **index.html**. The first line declares the application as a component so that it can be integrated into the Fiori Launchpad. Do not forget to replace 000 with your own number.

Since we transferred our application logic into a component, we have to edit the **index.html** file. Its solely purpose will be to call the component, further details, e.g. which resources have to be loaded, are from now on handled by the **Component.js** file.

Now open the **index.html** file. Delete the division inside of the body and give the id **"content"** to the body itself. Rewrite the app instantiation as shown below. Also change the script definition (the upper script-tag). Do not forget to replace 000 with your own number.

Now open the **index.html** file. Delete the division inside of the body and give the id **"content"** to the body itself. Rewrite the app instantiation as shown below. Also change the script definition (the upper script-tag). Do not forget to replace 000 with your own number.

```

<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta http-equiv='Content-Type' content='text/html; charset=UTF-8'/>
    <title>SAPUI5 Navigation</title>
    <script
      src="resources/sap-ui-core.js"
      id="sap-ui-bootstrap"
      data-sap-ui-theme="sap_bluecrystal"
      data-sap-ui-libs="sap.m"
      data-sap-ui-bindingSyntax="complex"
      data-sap-ui-compatVersion="edge"
      data-sap-ui-preload="async"
      data-sap-ui-resourceroots='{ "mvc_app_000": "." }'
    >
    </script>

    <script>

      sap.ui.getCore().attachInit(function () {
        sap.ui.require([
          "sap/m/Shell",
          "sap/ui/core/ComponentContainer"
        ], function (Shell, ComponentContainer) {
          new Shell({
            app : new ComponentContainer({
              name : "mvc_app_000",
              settings : {
                id : "webapp"
              }
            })
          }).placeAt("content");
        });
      });

    </script>

  </head>
  <body class="sapUiBody" id="content">
  </body>
</html>

```

Since we are going to create multiple controllers, which will share some basic functionalities, we can make use of the statements `.define` and `.extend` because they offer structure similar to inheritance in object-oriented programming. In our case the javascript-file **"baseController.js"** will contain those basic functions and can be extended by the other controllers. Create it in the controller folder. Do not forget to replace 000 with your own number.

```

sap.ui.define([
  "sap/ui/core/mvc/Controller",
  "sap/ui/core/routing/History"
], function (Controller, History) {
  "use strict";

  return Controller.extend("mvc_app_000.controller.BaseController", {

  });
});

```

In this last step, we want to apply the asynchronous module definition to our main.controller as well. Therefore, change the code of the controller accordingly and do not forget to replace 000 with your own number.

```

sap.ui.define([ "mvc_app_000/controller/BaseController" ], function(
    BaseController) {
    "use strict";

    return BaseController.extend("mvc_app_000.controller.main", {

        onInit : function() {

        }

    });
});

```

Adapt the controller name in the main view:

```
controllerName="mvc_app_000.controller.main"
```

Task 3: Implement the initial view

Short description: Inspecting the main view and creating the layout of it

Open the **main.view.xml** file that you've moved into the view folder and choose the **Source** tab, which is located at the bottom of the main window if you do not see the source code per default. There, we want to define a central tab view, which is shown when the application is started. In this exercise, we want to create an application which shows **Arrival** and **Departure** flights in two different tabs. For this, first change the title of the view to **Flight Monitor**:

```
<Page title="Flight Monitor">
```

Next, we want to create two tabs, which will be called **Arrival** and **Departure**. In the **Arrival** tab, we want to show a table which contains static flight data. The **Departure** tab will stay initially empty, and will only contain a small text line which announces that this feature will be implemented soon (actually, the content of the **Departure** tab will be part of the challenge at the end of this exercise). Therefore, to create the mentioned functionalities, first add the TabBar to your code (**content section** of the **main** view):

Implement initial view

```

<IconTabBar id="centralTabBar"
    expanded="{device>/isNoPhone}" class="sapUiResponsiveContentPadding">
    <items>

        </items>
</IconTabBar>

```

Define the two tabs inside if the TabFilter. Do not forget to save your file afterwards.

```

<IconTabFilter text="Arrival">
  <Table id="arrivingFlights" items="{/Flight}" growing="true"
    growingScrollToLoad="true">
    <columns>
      <Column>
        <Text text="Flight ID" />
      </Column>
      <Column>
        <Text text="From" />
      </Column>
      <Column>
        <Text text="To" />
      </Column>
      <Column>
        <Text text="Date" />
      </Column>
      <Column>
        <Text text="Time" />
      </Column>
      <Column>
        <Text text="Status" />
      </Column>
    </columns>
    <items>
      <ColumnListItem type="Navigation"
        press="handleListItemPress">
        <cells>
          <Text text="{flightId}" />
          <Text text="{from}" />
          <Text text="{to}" />
          <Text text="{date}" />
          <Text text="{time}" />
          <Text text="{status}" />
        </cells>
      </ColumnListItem>
    </items>
  </Table>
</IconTabFilter>
<IconTabFilter text="Departure"
  modelContextChange="onTabChange">
  <Text text="Here you will see departing flights soon!" />
</IconTabFilter>

```

main.view.xml:
<content>
section

As you can see, the **Arrival** tab contains a table definition, which links to **Flight** items.

Task 4: Implement routing

Short description: Add multiple views and navigate between them, using routing

The developed application is not runnable yet. Routing implementation is necessary for that. This enables us to navigate within a single-page application and bookmark pages or applications-states by adding a hash followed by a pattern at the end of the URL (url#/pattern).

To implement routing, we have to make some minor adjustments to the files we created so far. First, initialize a router by adding `this.getRouter().initialize();` to the init-function inside of Component.js.

Second, we need a few more views to make use of the router. We will demonstrate basic routing with a home-screen from which we can navigate to our main view. Create the view called **"enter"**. Do not forget to replace 000 with your own number.

```

<core:View xmlns:core="sap.ui.core"
  xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m"
  controllerName="mvc_app_000.controller.enter"
  xmlns:html="http://www.w3.org/1999/xhtml">
  <Page title="Flight Monitor">
    <content>
      <FlexBox alignItems="Center" justifyContent="Center"
        width="100%" height="100%">
        <Panel id="LoginPanel">
          <content>
            <Button text="Show flight list using navTo" press="onPress" />
            <Button text="Show flight list using getTargets"
              press="onDisplay" />
          </content>
        </Panel>
      </FlexBox>
    </content>
  </Page>
  <App id="enter" />
</core:View>

```

and the respective controller.

```

sap.ui.define([ "mvc_app_000/controller/BaseController" ], function(
  BaseController) {
  "use strict";

  return BaseController.extend("mvc_app_000.controller.enter", {
    onInit : function() {
    },
    onPress : function(evt) {
      this.getRouter().navTo("flightlist", {}, false);
    },
    onDisplay : function(evt) {
      this.getRouter().getTargets().display("flightlist");
    }
  });
});

```

In the code you already come across the two ways of triggering a routing process. In the first case, using `.navTo` you specify a target, data you want to transmit between the two views as well as a Boolean, indicating whether the current view should be excluded from the router-history. The `.display` function calls the view specified by the target, but won't add the according hash to the URL.

To navigate with the router history and to use the function `getRouter()` we have to implement two functions in our baseController. The required dependency has already been declared:

```

getRouter : function () {
  return sap.ui.core.UIComponent.getRouterFor(this);
},

onNavBack: function (oEvent) {
  var oHistory, sPreviousHash;

  oHistory = History.getInstance();
  sPreviousHash = oHistory.getPreviousHash();

  if (sPreviousHash !== undefined) {
    window.history.go(-1);
  } else {
    this.getRouter().navTo("home", {}, true /*no history*/);
  }
}

```

Now we are going to define the routes. Go to **manifest.json** and add the parenthesis **}** to overcome the error. Save the file. Add the object **sap.app** between the parenthesis, replacing the **###** in the id with your own number:

```
"sap.app": {
  "version": "1.1.0",
  "id": "mvc_app_###",
  "type": "application",
  "applicationVersion": {
    "version": "1.1.0"
  },
  "title": "Flight Monitor"
},
```

This entry defines the parameters that are necessary to register the app into the Fiori Launchpad. The next entry configures the root view, the json model (will be created in the next task), and the routing, replacing the 000 in the id with your own number:

```
"sap.ui5": {
  "rootView": {
    "viewName": "mvc_app_000.view.enter",
    "type": "XML",
    "async": true
  },
  "models": {
    "Flights": {
      "type": "sap.ui.model.json.JSONModel",
      "uri": "model/flights.json"
    }
  },
  "routing": {
    "config": {
      "viewPath": "mvc_app_000.view",
      "controlAggregation": "pages",
      "routerClass": "sap.m.routing.Router",
      "viewType": "XML",
      "transition": "slide",
      "async": true
    },
    "routes": [{
      "pattern": "",
      "name": "home",
      "target": "home"
    }, {
      "pattern": "flightList",
      "name": "flightList",
      "target": "flightList"
    }],
    "targets": {
      "home": {
        "viewId": "enter",
        "viewName": "enter",
        "controlId": "app",
        "viewLevel": 1
      },
      "flightList": {
        "viewId": "main",
        "viewName": "main",
        "controlId": "app",
        "viewLevel": 1
      }
    }
  }
}
```


Notice the way routes and their targets are declared. Depending on the URL we can now directly access our main-view (##flightList) or our enter-view (no hash after the URL). The root view that will be loaded when the user opens the application is the enter view. The control id that the views are embedded into is located in the App view. Create this **app.view** and the respective controller as seen below. Do not forget to replace 000 with your own number.

App.view.xml

```
<mvc:View
    controllerName="mvc_app_000.controller.App"
    xmlns="sap.m"
    xmlns:mvc="sap.ui.core.mvc"
    displayBlock="true">
    <App id="app"/>
</mvc:View>
```

App.controller.js

```
sap.ui.define([
    "mvc_app_000/controller/BaseController"
], function (BaseController) {
    "use strict";

    return BaseController.extend("mvc_app_000.controller.App", {
        onInit: function () {
        }
    });
});
```

Now that the app containing all the views has been created, it has to be instantiated in the Component.js. Add the following function to your component after the init function:

```
createContent: function(){
    var oViewData = {
        component : this
    };

    return sap.ui.view({
        viewName:"mvc_app_000.view.App",
        type: sap.ui.core.mvc.ViewType.XML,
        viewData: oViewData
    });
}
```

Your WebContent folder structure should now contain the following elements:

```

v WebContent
v controller
  > App.controller.js
  > baseController.js
  > enter.controller.js
  > main.controller.js
  > META-INF
  > model
v view
  App.view.xml
  enter.view.xml
  main.view.xml
  > WEB-INF
  > Component.js
  index.html
  { } manifest.json

```

Task 5: Integrate a static data model

Short description: Implement a basic part of the controller component and integrate a static data model provided by a JSON file.

For now, we have referenced the JSON data model multiple times, it is not created just yet. Static JSON model is used since connections to a database will only be handled in the **OData Services** chapter. Add the file **flights.json** under the **model** folder.

Your file opens. Please add the following data to the JSON file. Please pay attention to the right syntax, since the JSON syntax is very strict.

Create JSON data file

```

{
  "Flight": [
    {
      "flightId" : "LH1234",
      "from" : "Munich",
      "to" : "Frankfurt",
      "date" : "11.05.2013",
      "time" : "16.50",
      "status" : "On time",
      "plannedArrival" : "16:45",
      "info" : "Arrives early"
    },
    {
      "flightId" : "LH2345",
      "from" : "Frankfurt",
      "to" : "Montreal",
      "date" : "23.12.2015",
      "time" : "11.20",
      "status" : "Delayed",
      "plannedArrival" : "12:20",
      "info" : "Delayed due to technical problems"
    },
    {
      "flightId" : "LH3456",
      "from" : "Montreal",
      "to" : "New York",
      "date" : "23.07.2016",
      "time" : "12.30",
      "status" : "Canceled",
      "plannedArrival" : "CANCELED",
      "info" : "Canceled due to bad weather"
    },
    {
      "flightId" : "LH7890",
      "from" : "Dortmund",
      "to" : "Sharm-el-Sheikh",
      "date" : "26.08.2016",
      "time" : "12.30",
      "status" : "On time",
      "plannedArrival" : "09:00",
      "info" : "Arrives on time"
    }
  ]
}

```

Save your JSON file. Afterwards, open your controller **main.controller.js**. We want to load the data from the JSON file and show its contents in our table. For this, we have to modify the **onInit** function of the controller. Add the following code to the onInit Function (do not forget to replace 000 with your own number):

```
var sPath = $.sap.getModulePath("mvc_app_000", "/model/flights.json");
var oModel = new sap.ui.model.json.JSONModel(sPath);
this.getView().setModel(oModel);
```

Create model in controller

Save your controller. Now, we have created a static menu, which consists of two tabs. The first one shows a static table with three entries, while the second one only shows a small text line. The next step is to deploy the application to the SAP backend and to run it.

Task 6: Deploy your application to the SAP backend

Short description: Deploying the application to the SAP backend to make it accessible on the server

To deploy your application to the SAP backend, right-click on your web project **MVC_App** in the project explorer and choose **Team->Share Project**. In the next step, select **SAPUI5 ABAP Repository** as repository type. Afterwards, choose your system connection, and enter User and Password if necessary. Next, define to create a new BSP Application. Enter Name **Z_MVC_App_\$\$_###**, a description, and your Package to which you want to assign your application (e.g. **Z_\$\$_###**). Click **Next**.

Deploy application to SAP backend

Share Project

SAPUI5 Repository

Specify a BSP application

☐ Select a BSP Application:

Name	Description
/UI2/LAUNCHPAGE	Launchpage for Tile CHiPs
/UI2/TILECHIPS	CHiPs that can be used in Launchpage
/UI2/USHELL	Unified Shell
ARSRVC_SUITE_PB	Suite Page Builder
ARSRVC_UPB_ADMIN	Unified Shell Page Builder Administration Page

BSP Application:

☐ Create a new BSP Application

Name * Z_MVC_App_000

Description * Model-View-Controller App

Package * Z_000

Note:
Consider security aspects when using services via the Internet/Intranet, especially for administrative and development tasks.

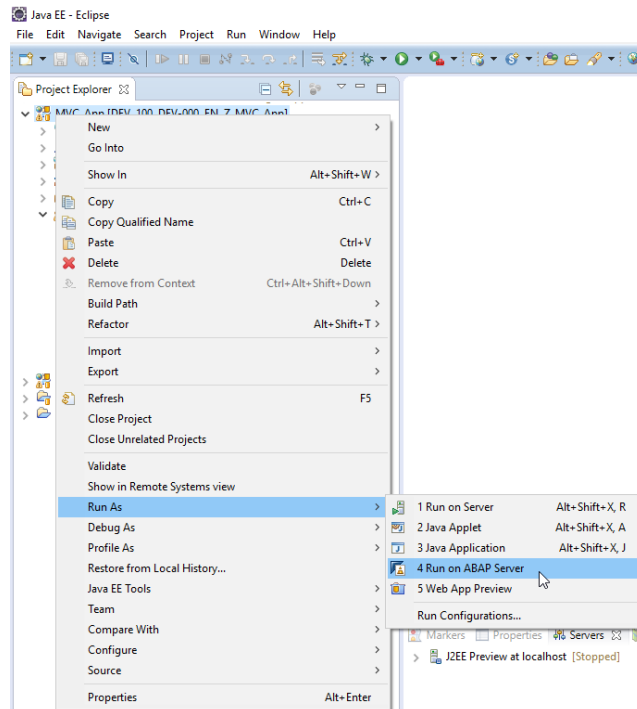
Choose a transport request. Again, you may use your default transport request you used for other exercises. Click on **Finish**. The application will be deployed to the SAP backend.

To test your application, first make sure that all source files were correctly submitted. For this, right-click on your project, and click on **Team->Submit**. Choose all your files and click on **Finish**. If **submit** is already greyed out, everything is fine.

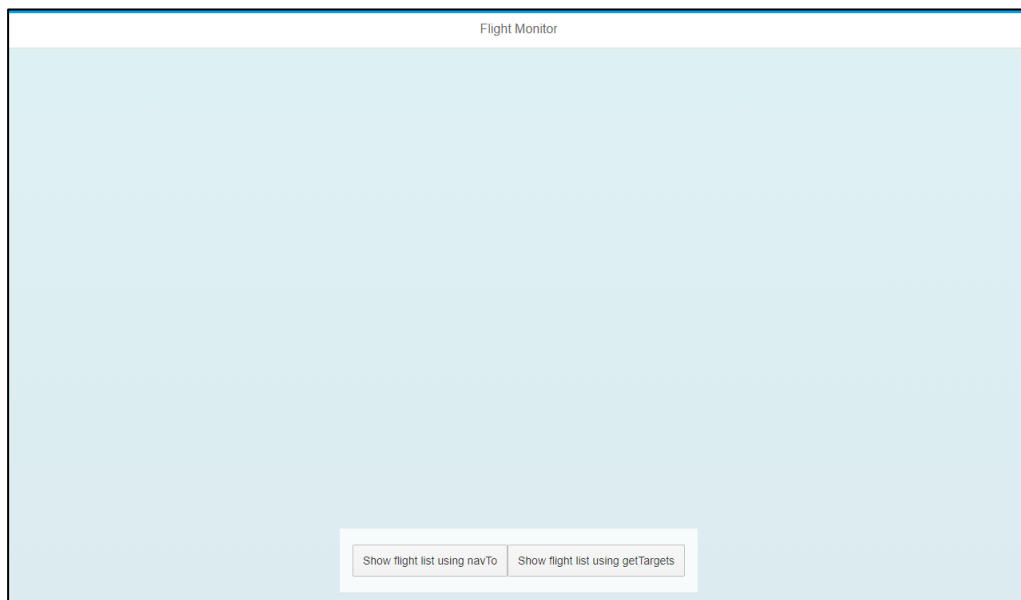
Submit repository objects

Finally, right-click on your project, and choose **Run As->Run on ABAP Server**. You may need to login with your username and password. If you test the application for the

first time, it may take some time for the window to retrieve all the information from the repository.



Afterwards, your application will be displayed in the integrated web browser (you can also copy the link to a local web browser and start your application there). If everything worked fine, it should look like this:



You can try out both ways of navigation, and you'll see the overview of the arrivals.

Flight Monitor					
Arrival Departure					
Flight ID	From	To	Date	Time	Status
LH1234	Munich	Frankfurt	11.05.2013	16.50	On time
LH2345	Frankfurt	Montreal	23.12.2015	11.20	Delayed
LH3456	Montreal	New York	23.07.2016	12.30	Canceled
LH7890	Dortmund	Sharm-el-Sheikh	26.08.2016	12.30	On time

For the departures you will see the placeholder string.

Flight Monitor					
Arrival <u>Departure</u>					
Here you will see departing flights soon!					

You can notice the different behavior of the “Back” button of your browser depending on which way of navigation you are using.

Task 7: Create a detail view for flight data

Short description: Creating a detail view, which shows detailed information about a selected flight from the table

In order to show detailed flight data after an entry of the flights table was selected, we have to create a new view and a corresponding controller. Name it **details.view** and implement it as follows (do not forget to replace 000 with your own number):

```
<core:View xmlns:core="sap.ui.core"
  xmlns:f="sap.ui.layout.form" xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m"
  controllerName="mvc_app_000.controller.details"
  xmlns:html="http://www.w3.org/1999/xhtml">
  <Page title="Flight Monitor">
    <content>
      <VBox class="sapUiSmallMargin">
        <f:SimpleForm id="flightDetails" title="Flight Details">
          <f:content>
            <Label text="Flight ID" />
            <Text text="{flightId}" />
            <Label text="From" />
            <Text text="{from}" />
            <Label text="To" />
            <Text text="{to}" />
            <Label text="Date" />
            <Text text="{date}" />
            <Label text="Time" />
            <Text text="{time}" />
            <Label text="Status" />
            <Text text="{status}" />
            <Label text="Planned Arrival" />
            <Text text="{plannedArrival}" />
            <Label text="Information" />
            <Text text="{info}" />
          </f:content>
        </f:SimpleForm>
      </VBox>
    </content>
  </Page>
  <App id="details" />
</core:View>
```

Enter the code below to the **details.controller** (do not forget to replace 000 with your own number):

```
sap.ui.define([ "mvc_app_000/controller/BaseController" ], function(
    BaseController) {
    "use strict";
    return BaseController.extend("mvc_app_000.controller.details", {
        onInit : function() {
            var sPath = $.sap.getModulePath("mvc_app_000", "/model/flights.json");
            var oModel = new sap.ui.model.json.JSONModel(sPath);
            var oView = this.getView();
            oView.setModel(oModel);
            this.getOwnerComponent().getRouter().getRoute("flights").attachPatternMatched(this._onRouteMatched,
                this);
        },

        _onRouteMatched: function(oEvent) {
            var flightId = oEvent.getParameter("arguments").flightId;
            var oView = this.getView();
            var sIdPrefix = oView.getId().split("---")[0];
            this.getView().addDelegate({
                onBeforeShow : function(evt){
                    var oArray = sap.ui.getCore().byId(sIdPrefix+"---main").getModel().getData().Flight
                    function matchById(obj){
                        return obj.flightId === flightId;
                    }
                    var index = oArray.findIndex(matchById);
                    oView.bindElement("/Flight/"+index);
                }
            });
        }
    });
});
```

Next, we have to link to the detail page after a user has clicked on a table entry in the **main** view. You may have probably noticed that in task 2, we assigned the **handleLineItemPress** function to the **press** event of the **ColumnListItem**. We now want to define this function, in which we link to the detail page, and pass the flightID of the object the user has selected. Therefore, please add the following code to the **main.controller.js** at the end of the file:

Handle press event

```
handleLineItemPress: function (oEvent) {
    var context = oEvent.getSource().getBindingContext();
    console.log(context.getProperty("from"));
    this.getRouter().navTo("flights",{
        flightId : context.getProperty("flightId")
    });
}
```

In the last step, we will add the pattern for the details.view. Go to manifest.json and add the route and target:

```
{
    "pattern": "flights/{flightId}",
    "name": "flights",
    "target": "flights"
}
```

```
"flights": {
    "viewId": "details",
    "viewName": "details",
    "controlId": "app",
    "viewLevel" : 2
}
```

To test your application, submit the changes to the backend (**Team->Submit**) and rerun it (**Run As->Run on ABAP Server**).

When you now click on one of the flight objects in **Arrival** tab, you will be directed to the detail page. The flightID is also part of the URL of the details.view.

Flight Details	
Flight ID:	LH1234
From:	Munich
To:	Frankfurt
Date:	11.05.2013
Time:	16:50
Status:	On time
Planned Arrival:	16:45
Information:	Arrives early

Task 8: Debug your application

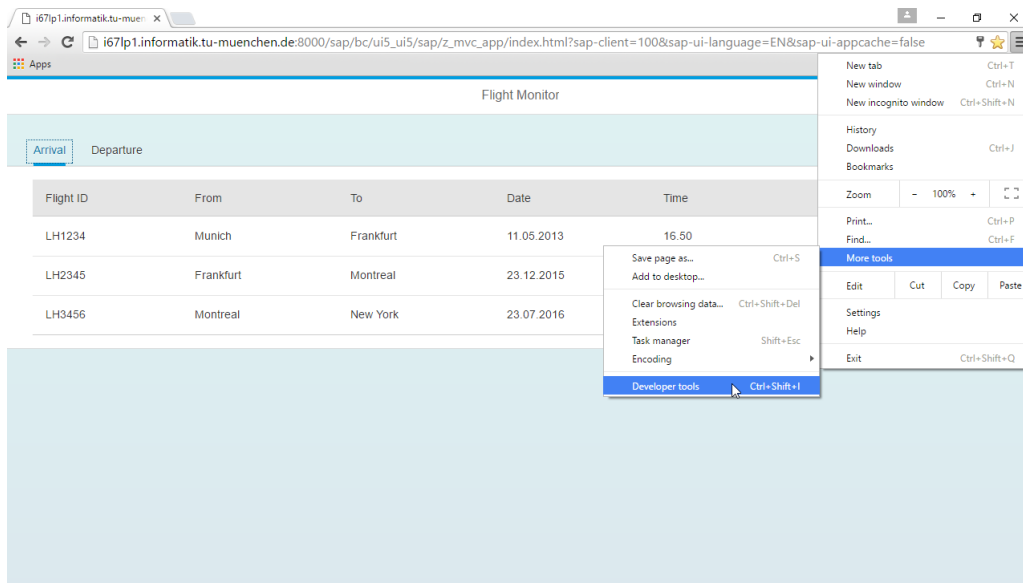
Short description: Debugging the application using the integrated developer tools of a web browser (e.g. Google Chrome)

Debugging is an important task to find errors in your program and retrace the program execution. In many programming languages, the debugger is integrated in the IDE. However, since JavaScript is only running in a browser together with HTML, you need the help of the browsers' development tools to debug an application. When you debug your program, in most of the cases, you want to retrace your variables and the value of attributes at a certain point of the program. For this, you can set breakpoints, which stop a program at the respective point. In this exercise, we will demonstrate this by setting a breakpoint after a flight element was clicked in the table, and showing which variables are passed between the views. For this, go to the main controller, and add the following line right BEFORE the **var context = evt.getSource().getBinding Context();** statement:

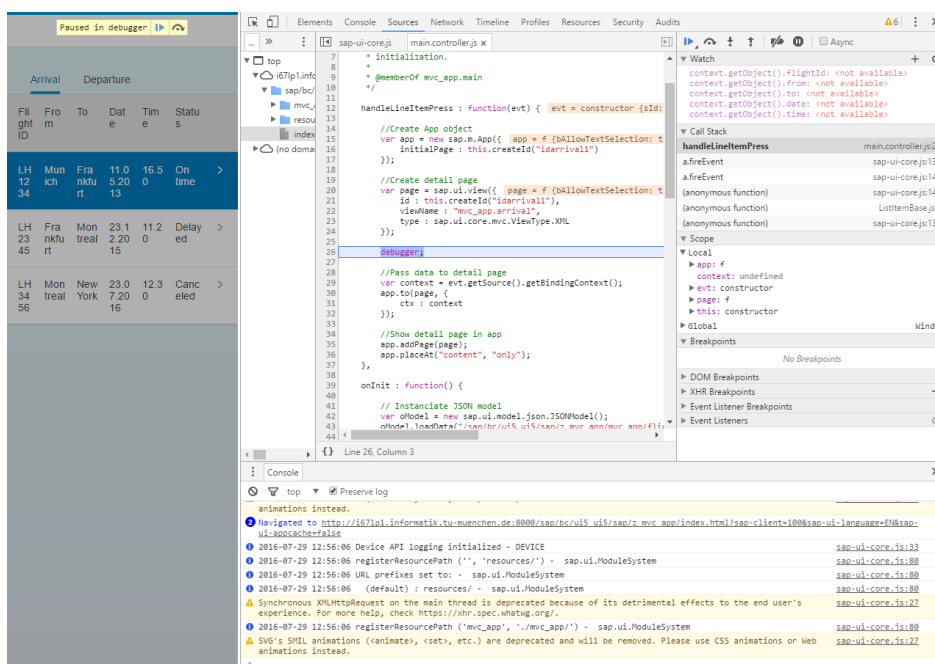
Set breakpoint

```
debugger;
```

Save the file, and submit your changes to the SAP backend by clicking **Team->Submit** in the context menu of your project. Now, right-click on your project, and choose **Run As->Run on ABAP Server**. The integrated browser opens. Please copy the link at the top of the browser window into a local browser, preferably Google Chrome. You may need to login with your username and password. Open the developer tools by clicking on the menu on the upper right and choosing **More Tools->Developer Tools**.



Your application will be shown together with the developer tools. There, a central tool to debug errors is the console. The console shows warnings and system errors in a list. For many errors in your application, the console already shows useful hints on how to solve them. However, in some cases, you have semantic errors which are not shown in the console. For this, it is helpful to know how your variables are changed by your program. In our case, we have already set a breakpoint to debug the variables of our flight table right before the detail page is opened. Therefore, to start the debugger, click on a table entry. The developer tool will jump to the line in the source code where you have set the breakpoint.



Please search for your variable **context** a few lines below the debugger statement. Double-click on it to select it, right-click, and choose **Add to watch**. Your variable will be added to the **watch** area in the upper right of your developer tools.

Debug variables

Double-click on **context** in the **Watch** area, and change the name from **context** to **context.getObject().flightId**. Thereby, you get access to the current value of your variable **flightId**, which should be empty. Please add some more variables of your flight object by clicking on **+**. It should look like this:


```
context.getObject().flightId: <not available>
context.getObject().from: <not available>
context.getObject().to: <not available>
context.getObject().date: <not available>
context.getObject().time: <not available>
```

You see that there are currently no values assigned to the **context** object, because the variable was not yet created. Therefore, execute your program step-by-step by clicking

on  (or pressing F10) above the **watch** section four times. Now, the variable was created, and you should see the values of your selected flight object.

```
context.getObject().flightId: "LH1234"
context.getObject().from: "Munich"
context.getObject().to: "Frankfurt"
context.getObject().date: "11.05.2013"
context.getObject().time: "16.50"
```

Go on with the program execution by clicking on . Close the development tools by clicking on .

Remark: Although we only traced some single variables in this exercise, you may have probably noticed that the debugger is a powerful tool to find errors. For example, if a text field does not show the right value, you can always trace how a variable value changed, and why it probably shows the wrong value. By adding values to the **watch** section, you can simulate program executions, and find out e.g. how you have to change your code. Furthermore, strong errors such as syntax errors are directly shown in the console. Therefore, always make use of the debugger if you want to find out more details about your program logic.

Challenge: Implement the Departure tab of your SAPUI5 application

Task 1: Implement the departure tab, similar to how it was done for the arrival tab. Create a new JSON file, in which you change the **plannedArrival** attribute to **plannedDeparture**. Then, manipulate the main view to show a table in the departure tab with this flight data, having the same columns as in the arrival tab. You also have to load the new JSON model in the **onInit** function. Afterwards, create a new detail view called **departure** for departing flights, containing the **plannedDeparture** field instead of **plannedArrival**. Finally, implement the **press** event and a back button using the **departure** view and its controller. Submit your changes to the SAP backend.

Task 2: Debug your application using a breakpoint in the **onBeforeShow** function, which is called within the **onInit** function of your new **departure** controller. Investigate the **evt.data.ctx** variable by displaying if the right flight data is stored therein. For this, add the variable to the watch section and modify its expression accordingly to show the single flight attributes (cf. task 6 of the exercise).